COP2034C Module 10 Programming Assignment

Instructions

In this assignment, we will continue developing our Palmer Penguins program from Module 9. Previously, we introduced object-oriented programming by creating a Penguin class, storing penguin observations as objects, and using object attributes to calculate body mass statistics.

In Module 10, we will introduce exception handling to make our program more reliable when encountering invalid input or problems accessing data. Instead of allowing certain errors to terminate the program, we will use Python's exception-handling features to display meaningful messages and continue processing when appropriate.

We will practice selected Chapter 10 concepts:

- Handling exceptions using try and except.
- Handling different exception types using multiple except blocks.
- Raising exceptions using raise.
- Handling exceptions raised within functions.
- Using finally to perform an action regardless of whether an exception occurs.

1. Start with Your Module 9 Solution

Open your completed Module 9 notebook in Google Colab and make a copy for Module 10. Suggested notebook name:

```
PalmerPenguinsM10.ipynb
```

The downloaded Python program should be named:

```
PalmerPenguinsM10.py
```

Update the ID header:

```
# PalmerPenguinsM10.py
# Your Name
# Submission Date
# Analyze Palmer Penguin observations using exception handling
```

Retain the working functionality from Module 9, including the existing functions, species statistics, CSV processing, body mass summaries, dictionary operations, Penguin objects, and interactive species exploration. We will add exception handling to selected portions of the program without removing the earlier demonstrations of Python concepts. The penguin.py module does not require any changes for this assignment.

2. Handle Errors When Reading the CSV File

Our existing read_penguin_row() function opens the Palmer Penguins CSV file and retrieves a specified observation.

Currently, if the CSV file cannot be found, Python raises a FileNotFoundError exception and terminates the program. We will modify this function to handle that exception.

Locate the existing function:

```
def read_penguin_row(row_number):
```

Keep the existing logic, but place the file-reading statements inside a try block. Add an except FileNotFoundError block that displays:

```
Error: PalmerPenguins.csv was not found.
```

Use the following structure:

```
def read_penguin_row(row_number):
    penguin_row = ""
    try:
        # existing code that opens and reads the CSV file
    except FileNotFoundError:
        print("Error: PalmerPenguins.csv was not found.")
    return penguin_row
```

Replace the comment with the existing file-reading logic, properly indented inside the try block.

Notice that the function still uses one return statement at the end, following the convention established in previous modules.

Important: This change handles the missing-file exception, but the remainder of the program still expects valid CSV data. We will address that situation in the next step.

3. Raise an Exception for a Missing Observation

Sometimes a CSV file exists, but the requested observation cannot be found.

Our read_penguin_row() function currently returns an empty string when the requested row does not exist. If the program subsequently attempts to process that string as CSV data, an error may occur.

We will use raise to identify this problem explicitly.

Inside read_penguin_row(), after the existing file-reading logic and its exception handler, add a condition that checks whether penguin_row is empty.

If it is empty, raise a ValueError with the following message:

```
raise ValueError(f"Penguin observation {row_number} "
                 f"could not be retrieved.")
```

Place this condition before the final return statement. This will handle two situations:

- The requested observation number does not exist in the CSV file.
- The CSV file could not be opened, leaving penguin_row empty.

A raise statement raises an exception that can be handled by the calling function. In the next step, we will demonstrate this behavior.

4. Handle Exceptions Raised by a Function

In Chapter 10, we learned that an exception raised inside a function can be handled by the function that called it.

We will demonstrate this using our existing process_penguin_row() function.

Locate:

```
def process_penguin_row(row_number):
```

This function retrieves a CSV observation, splits the data into individual fields, converts numeric values, and displays the observation. Modify the function to place its existing statements inside a try block and add an except ValueError block to display the exception message:

```
except ValueError as excpt:
    print(f"Error: {excpt}")
```

Also add an except IndexError block:

```
except IndexError:
    print("Error: The penguin observation is incomplete.")
```

The two handlers address different problems.

Exception	Situation
ValueError	A requested observation cannot be retrieved, or a value cannot be converted into a number.
IndexError	A CSV observation does not contain enough fields to access the expected data.

The **as excpt** syntax allows us to access and display the message associated with a ValueError.

Important: Do not remove the existing processing statements. The new try block should contain the original functionality.

5. Demonstrate Exception Handling with an Invalid Observation

We will now test the exception handling added to our CSV processing functions.

Inside main(), locate the existing calls:

```
process_penguin_row(1)
process_penguin_row(153)
```

Immediately after these calls, add:

```
print("\nException Handling Demonstration")
process_penguin_row(999)
```

Our dataset contains 342 observations, so observation 999 does not exist.

The read_penguin_row() function should raise a ValueError, which will be handled by process_penguin_row(). The program should display:

```
Exception Handling Demonstration
Error: Penguin observation 999 could not be retrieved.
```

The program should then continue executing the remaining sections. This demonstrates how exception handling allows a program to respond to a problem without terminating unexpectedly.

6. Add Exception Handling to Interactive Species Exploration

Our existing interactive species exploration allows the user to enter a species name and view its statistics. We will retain that functionality and add a new option to retrieve a penguin observation by its sample number.

Inside main(), locate the existing interactive species exploration section.

Update the introductory message to display:

```
Explore the Palmer Penguins
Enter Chinstrap, Gentoo, Adelie, Sample, or Done.
```

The existing species selections should continue working as before. When the user enters Sample, the program should prompt for a sample number and attempt to display the corresponding observation.

Add the Sample option

Inside the existing while loop, modify the decision structure to recognize Sample as an additional valid command.

Use the following structure:

```
if species_choice == "Sample":
    # prompt for a sample number
    # convert the input to an integer
    # call process_penguin_row()
elif species_choice not in species_names:
    print("That is not a valid species.")
else:
    # existing species exploration logic
```

The Sample option should not increase valid_selections, since that counter represents valid species selections.

Handle invalid numeric input

Inside the Sample branch, use a try block to convert the user's input to an integer using this structure:

```
try:
    sample_number = int(input("Enter a sample number: "))
    process_penguin_row(sample_number)
except ValueError:
    print("Error: Please enter a valid integer sample number.")
```

If the user enters text such as abc, the conversion will raise a ValueError. The exception handler will display the message, and the program should return to the existing species exploration prompt.

If the user enters a valid integer that does not correspond to an observation, process_penguin_row() will display the appropriate error message. The program should continue accepting commands until the user enters Done.

7. Use finally to Confirm Completion of a Sample Request

The finally clause allows a program to execute statements regardless of whether an exception occurred. We will use one to display a completion message after each sample request. Return to the try and except structure added in Step 6.

Add a finally block that displays:

```
finally:
    print("Sample request completed.")
```

This message should appear after each sample request, whether the user entered a valid integer or invalid text. For example:

```
Enter a species: Sample
Enter a sample number: abc
```

```
Error: Please enter a valid integer sample number.
Sample request completed.
```

The finally clause is often used to perform cleanup operations, such as closing files or releasing resources. In this assignment, we are using it to demonstrate that certain statements execute regardless of whether an exception occurs.

8. Organize the Updated Program

The Module 10 changes should be integrated into the existing program rather than added as an entirely separate application. Your program should follow this general sequence:

```
main()
    display species statistics
    process two sample observations
    demonstrate an invalid observation
    collect and summarize body masses
    display species dictionary

    create one Penguin object
    display the object's information

    create a list of five Penguin objects
    display each object
    calculate object-based body mass statistics

    explore species using the existing while loop
        process valid species selections
        process Sample requests
        handle invalid sample input
        display completion message

    display final exploration summary
```

Keep the original Module 9 functionality working. The new exception-handling features should be incorporated into the existing functions and interactive loop.

Expected Output

The program should continue displaying the existing Module 9 output, including species statistics, CSV observations, body mass summaries, Penguin objects, and interactive species exploration. The new Module 10 output should include the following demonstrations.

Invalid CSV observation

```
Exception Handling Demonstration
```

```
Error: Penguin observation 999 could not be retrieved.
```

Valid sample request

```
Explore the Palmer Penguins
Enter Chinstrap, Gentoo, Adelie, Sample, or Done.

Enter a species: Sample
Enter a sample number: 1

Sample dataset row:
1,Adelie,39.1,18.7,181,3750,MALE
['1', 'Adelie', '39.1', '18.7', '181', '3750', 'MALE']

Sample Penguin Observation
Sample Number: 1
Species:       Adelie
Bill length:   39.1 mm
Bill depth:    18.7 mm
Body mass:     3750 g
Sex:           MALE
Sample request completed.
```

Invalid numeric input

```
Enter another species: Sample
Enter a sample number: abc
Error: Please enter a valid integer sample number.
Sample request completed.
```

Observation not found

```
Enter another species: Sample
Enter a sample number: 999
Error: Penguin observation 999 could not be retrieved.
Sample request completed.
```

Existing species exploration

```
Enter another species: Adelie

Species: Adelie
Count: 151
Percentage: 44.15%
```

```
This species represents a large portion of the dataset.
It is the largest species in the dataset.
Its count is above the average.

Enter another species: Done

You explored 1 valid penguin species.
```

The output above uses the provided Palmer Penguins CSV file. Minor whitespace differences are acceptable as long as the information is correct and readable.

Test Your Program

Before submitting, verify that:

- The original Module 9 functionality still works correctly.
- The read_penguin_row() function handles a missing CSV file using FileNotFoundError.
- The function raises a ValueError when an observation cannot be retrieved.
- The process_penguin_row() function handles ValueError and IndexError exceptions.
- The invalid observation demonstration displays an error message without terminating the program.
- The new Sample option retrieves and displays a valid observation.
- Entering text instead of a sample number displays an appropriate error message.
- Requesting a nonexistent observation displays an appropriate error message.
- The finally block displays its completion message after every Sample request.
- The existing species exploration still works, and entering Done ends the loop.
- The penguin.py module continues working without modification.

Data Science Connection

In previous modules, we used CSV files, lists, dictionaries, and objects to organize and analyze Palmer Penguins observations. In Module 10, we introduce an important consideration in data science: handling errors that occur while accessing and processing data.

Real-world datasets are not always complete or correctly formatted. A data file may be missing, a numeric field may contain unexpected text, or an observation may not contain all the expected values. If a program assumes that every observation is valid, these problems can interrupt an analysis or produce unreliable results.

Exception handling provides a way to recognize and respond to these situations. Our updated program demonstrates several practices used in data-processing applications:

- File access: Detect when a required dataset cannot be found.
- Data retrieval: Identify when a requested observation does not exist.
- Data conversion: Handle values that cannot be converted to the expected numeric type.
- Data structure validation: Detect when an observation does not contain the expected fields.

- Processing continuity: Allow the program to continue after certain errors have been handled.

For example, a data scientist processing thousands of observations may encounter records containing missing or invalid measurements. Instead of allowing one problematic record to terminate the entire process, the program can identify the problem, report it, and continue processing other observations when appropriate.

Exception handling does not automatically correct inaccurate data. Rather, it helps developers identify problems and determine how their programs should respond. Our Palmer Penguins program now demonstrates a more reliable data-processing workflow by combining file operations, object-oriented programming, and exception handling.

Readability

Continue following the coding conventions established in previous modules:

- Use meaningful variable and function names.
- Use consistent four-space indentation.
- Include comments describing important operations.
- Use logical blank lines to separate program sections.
- Keep exception handlers close to the statements that may produce the exceptions.
- Use specific exception types rather than a general except: statement.
- Preserve the existing organization of the main program and the custom penguin.py module.

Exception handling should make the program easier to understand by separating normal processing statements from statements that respond to errors.

Screenshot and Submission

Run the completed program in Google Colab. Capture enough of the execution output to demonstrate the new exception-handling functionality, including an invalid observation, a valid Sample request, an invalid numeric input, and the existing interactive species exploration.

Download the following files from Google Colab:

Filename	**Description**
PalmerPenguinsM10.py	Main Python program containing the new exception-handling functionality
penguin.py	Custom module containing the Penguin class, retained from Module 9
PalmerPenguinsM10.png	Screenshot of the program's execution output

To download the main Python program, use the appropriate Google Colab download option. Make sure the %%writefile penguin.py command is not included in the exported program, since it is Colab-specific and is not valid Python syntax.

To download penguin.py, open the Files panel on the left side of Colab. Locate the file, click the three-dot menu next to its filename, and select Download.

Upload all three files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.